Prepared for submission to JHEP

dSIBP: a package for generating IBP seeds for QFT in de Sitter spacetime

Jiaqi Chen[a,b]

[a] *Beijing Key Laboratory of Optical Detection Technology for Oil and Gas, China University of Petroleum-Beijing, Beijing 102249, China*

[b] *Basic Research Center for Energy Interdisciplinary, College of Science, China University of Petroleum-Beijing, Beijing 102249, China*

E-mail: jiaqichen@cup.edu.cn

Abstract: Integration-by-parts (IBP) relations and differential equations provide a systematic approach to Feynman integrals in de Sitter spacetime. `dSIBP` is a Mathematica package that makes this approach available from a graph declaration. For families with massive and massless propagators, it generates canonical IBP seeds, assembles linear systems for external reduction, and constructs differential equations for the reduced master integrals. The implementation combines equations of motion, contact terms and parity constraints, and provides Euler scaling checks. A complementary tree-level workflow implements iterative reduction and direct $d\log$ differential equations for supported time-only families. We describe the conventions and public interface through worked examples, including a massive bubble and a mixed loop-plus-tree family with eighty-one master integrals.

Contents

1 Introduction **1**

2 Notation and conventions **2**
- 2.1 Schwinger–Keldysh Feynman rules 3
- 2.2 The integral family 4
- 2.3 Kinematics 7
- 2.4 Parity structure 8
- 2.5 Massive function systems 8
- 2.6 Tree-level and time-only families 8

3 Using the package **9**
- 3.1 Initialisation 9
- 3.2 Seeds and the IBP index ranges 10
- 3.3 Linear systems and external reduction 11
- 3.4 Differential equations and checks 13
- 3.5 Tree-level track 14

4 Examples **14**
- 4.1 Validation 17

5 Summary and outlook **19**

A Public functions **20**

1 Introduction

Correlation functions in de Sitter (dS) spacetime describe the quantum fluctuations relevant to inflation. In the Schwinger–Keldysh formulation, their Feynman graphs involve integrations over vertex times and, at loop level, spatial loop momenta [1]. Massive propagators introduce Hankel functions into these integrals. Generalised integration-by-parts (IBP) relations and differential equations (DE) offer a systematic way to organise this calculation [2–4].

In flat-spacetime QFT, IBP relations reduce integral families to master integrals [5, 6]. Differentiating these masters and reducing the result gives closed differential equations in the kinematic variables [7–10]. A suitable basis can simplify the equations, in particular to canonical $d\log$ form [11]. This approach is supported by reduction programs such as FIRE [12, 13] and Kira [14, 15], rational-function reconstruction tools such as Rational Tracer [16], and numerical DE solvers such as DiffExp [17]. The auxiliary mass flow method and

its implementation AMFlow provide another example of turning reduction and differential equations into an automated numerical calculation [18–20].

For dS integrals, ref. [2] derives general IBP reduction formulas and $d\log$ differential equations for vertex integral families with Hankel functions. Reference [3] develops power-series solutions, multivariate hypergeometric representations and boundary conditions for vertex integral families and the homogeneous parts of tree integrals. At loop level, ref. [4] combines time and momentum IBP relations and organises the system into subsystems labelled by the parities of propagator indices. Its massive bubble example illustrates the reduction to master integrals and the construction of $d\log$ differential equations. Together these results provide the basis for a systematic computational approach to dS integrals. Applying it to a graph requires consistent treatment of the endpoint states of each propagator, the contact terms from time ordering, and the kinematic derivatives.

`dSIBP` automates this construction. Starting from vertices, propagators and momentum assignments, it builds an integral family and generates canonical IBP seeds. Equations of motion reduce massive endpoint states, algebraic relations identify massless states, and parity constraints restrict the generated system. The user chooses the ranges of integral indices and passes the resulting linear system to an external reducer. After importing a reduction, the package constructs differential equations for a chosen master basis and checks Euler scaling identities.

For loop families, the linear systems are solved externally; `dSIBP` does not launch Kira or provide a numerical solver for the resulting DE. A separate time-only workflow implements internal iterative reduction and direct $d\log$ equations for supported tree families. Its current scope is specified in section 2.6. Comparisons with analytic results and with MadStree [21] test the implementation as described in section 4.1.

Section 2 introduces the integral representation and the conventions needed for graph declarations. Section 3 follows the calculation from initialisation to differential equations. Section 4 presents the examples and validation results, and section 5 discusses further work. Appendix A lists the public functions.

Statement on AI use. The authors designed the notation, functionality, interfaces and validation tasks of `dSIBP`. Several agent programs, each driven by a different large language model, implemented the code and documentation under the authors' supervision. The validation combines comparisons with analytic results and separately constructed calculations with internal consistency checks. Section 4.1 distinguishes the tests of IBP relations, differential equations and scaling identities.

The package is available at `https://github.com/J7C/dSIBP-Packages`.

2 Notation and conventions

A graph declaration fixes the integral family on which the IBP and derivative operators act. This section specifies the propagator states, index conventions and kinematic variables needed to interpret that family and its contact sectors.

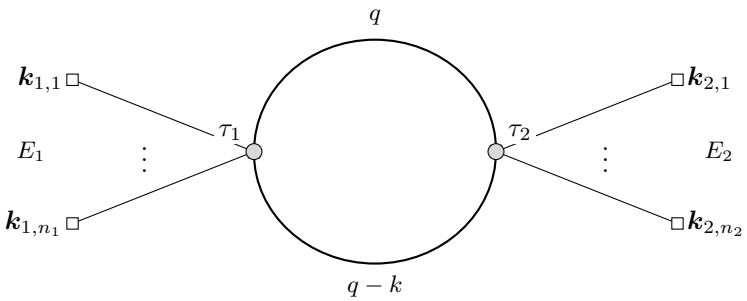


Figure 1. Feynman rules of a de Sitter bubble with general branch signs, drawn as grey vertices. The vertex energies E_1, E_2 and the conformal times τ_1, τ_2 are written at the vertices. The external legs end in open squares and carry the momenta $k_{1,1}, \ldots, k_{1,n_1}$ and $k_{2,1}, \ldots, k_{2,n_2}$. The two internal propagators carry the loop momenta q and $q-k$.

2.1 Schwinger–Keldysh Feynman rules

A de Sitter Feynman graph in the in-in formalism consists of vertices v with conformal time $\tau_v < 0$ and Schwinger–Keldysh branch signs $\sigma_v = \pm$. Each vertex with external energy E_v carries the phase factor

$$e^{-\mathrm{i}\sigma_v E_v \tau_v}\,, \tag{2.1}$$

and the time integrations run over the full interval $\tau_v \in (-\infty, 0)$. An internal propagator e joining the ordered vertices u and v is one of four types. It is massive or massless, and it is *full* when the two branch signs agree, $\sigma_u = \sigma_v$, and *cross* when they differ, $\sigma_u = -\sigma_v$. These are the propagators $G_{\sigma\sigma}$ and $G_{\sigma,-\sigma}$ of refs. [2–4]. Only a full propagator contains a Heaviside ordering of the two vertex times, so only a full propagator produces θ-shrink terms. The assignments $++$ and $--$ belong to the same full class, and $+-$ and $-+$ to the same cross class. Their kernels retain the branch dependence; equality of classes does not imply equality of propagators. In our figures a circle denotes a vertex, and its fill colour encodes the branch sign: black is $\sigma_v = +$, white is $\sigma_v = -$, and grey leaves the sign general. The conformal time τ_v is written below the vertex and the vertex energy beside it. External legs end in small open squares. Each vertex carries one energy E_v, the energy that appears in its phase factor. It is a symbol of the declaration: the user may bind it to the packaged magnitudes of the external legs attached to v, or keep it independent even when no external leg is attached to it. Figure 1 shows these rules for the bubble input of the package. Its two vertices are drawn in grey because their branch signs are general. Vertex 1 carries the external momenta $k_{1,1}, \ldots, k_{1,n_1}$ and vertex 2 carries $k_{2,1}, \ldots, k_{2,n_2}$. The internal momenta are q and $q-k$.

The package distinguishes the four propagator types as $\langle$`massive, massless`$\rangle \times \langle$`full, cross`$\rangle$. Figure 2 shows the resulting graphical conventions. Every propagator that lies on a cycle becomes a denominator of the integral family; the propagators on cycles form the *cycle skeleton* of the graph. A propagator that does not lie on a cycle is a *fixed propagator*, which the package also calls a *bridge*. It carries only a magnitude power and no loop-momentum denominator. Massive propagators whose two vertices become coincident

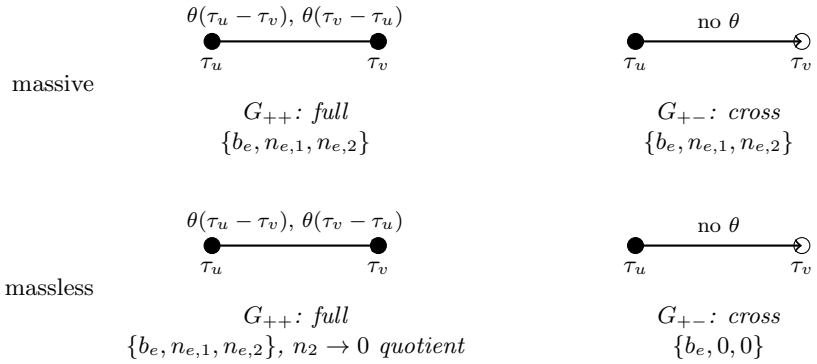


Figure 2. The four types of internal propagator and their label packs. The fill colour of a vertex encodes its branch sign: black is + and white is −. The conjugate assignments −− and −+ give the same two types. Massive full, massive cross and massless full propagators carry $\{b_e, n_{e,1}, n_{e,2}\}$, with the massive vertex states restricted to $\{0, 1\}$. A massless cross propagator is a single exponential product, so it carries $\{b_e, 0, 0\}$. A massless full propagator has two independent states with canonical second label zero.

under a contact operation may be *shrunk*. A shrink removes the time-dependent block and merges its endpoints; its residual magnitude powers and normalisation are retained. A fixed propagator retains its block until it is shrunk, while its magnitude power belongs to the sector normalisation N_s defined in section 2.2.

2.2 The integral family

A sector s specifies which propagator blocks remain and which have been shrunk by contact terms. Its active vertices are the vertices remaining after the corresponding identifications. Suppressing constant integration-measure conventions, a family member has the form

$$J_s = N_s \int \prod_{\ell=1}^{L} d^d q_\ell \int_{-\infty}^{0} \prod_{v \in V_s} d\tau_v \prod_{v \in V_s} (-\tau_v)^{A_v} e^{-\mathrm{i}\sigma_v E_v \tau_v} \times \prod_{e \in \mathcal{C}} \xi_e^{-B_e} \prod_{e \in U_s} \mathcal{B}_e^{(n_{e,1}, n_{e,2})}(\xi_e; \tau_u, \tau_v) \prod_r \rho_r^{i_r} \,. \tag{2.2}$$

Here d is the spatial dimension, q_ℓ are loop momenta, and V_s is the set of active vertices. The set $\mathcal{C}$ contains the cycle propagators of the declared graph, with momentum magnitudes ξ_e. The set U_s contains the unshrunk propagators, including fixed propagators. For $e = (u, v)$, $\mathcal{B}_e^{(n_{e,1}, n_{e,2})}$ is the corresponding Schwinger–Keldysh block. Its superscripts label endpoint states, not powers of the block: for a massive propagator, states 0 and 1 select a mode function and its first derivative with respect to its argument. Massless states are specified below. The ISPs ρ_r and their integer powers i_r are defined in section 2.3.

The time powers are $A_v = a_v + a0_v$. For an unshrunk cycle propagator, the momentum denominator power is $B_e = b_e + b0_e$; after a shrink it is the corresponding sector power $b_{S,e} + b_{S0,e}$. The zero points $a0_v$, $b0_e$ and their child-sector values are part of the declaration and its contact rules. The factor N_s collects the powers of no-loop magnitudes and

the normalisation constants associated with contractions. Fixed propagators contribute magnitude powers through this factor. Residual cycle-momentum powers remain under the loop integral. Kinematic derivatives act on N_s as well as on the remaining integrand. In the package the family is written in the three-slot schema

```
J[aList, linePacks, ispList]
```

The first slot is the list of vertex time powers a_v of the vertices that are still distinct in this sector, the *active* vertices of the sector, in the frozen vertex order. The second slot holds one pack per propagator, in the order in which the declaration key `"lines"` lists them, and the shapes of the packs identify the sector. The third slot lists the ISPs with their integer powers i_r. A case is one named declaration. Its keys give the vertices with their branch signs, the propagators with their endpoints, momenta and mass types, the external legs, the loop momenta, and the two lists of external momenta. They also carry the zero points, the symmetry rules and the parity constraints discussed below.

- A cyclic massive full, massive cross or massless full propagator carries $\{b_e, n_{e,1}, n_{e,2}\}$: the power b_e and the two state powers. The three integers are the propagator indices of refs. [2–4]; in this paper they are called the propagator power and the state powers. The latter name denotes endpoint-state indices, not ordinary powers of a function.
- A cyclic massless cross propagator contributes $\{b_e, 0, 0\}$. It is a single exponential product, so it has no discrete state.
- A fixed propagator contributes $\{\texttt{"F"}, n_{e,1}, n_{e,2}\}$: no propagator power, and its magnitude power is collected in N_s.
- A shrunk propagator contributes $\{b_{S,e}\}$, or $\{\texttt{"F"}\}$ when it was already fixed. The cycle-momentum power remains in the loop integrand; no-loop magnitude powers are collected in N_s.

Massive vertex states and equations of motion. Every massive propagator is assigned a second-order function system $f'' + P(x)f' + Q(x)f = 0$. Its variable is $x = -\xi\tau$, the negative of the product of the momentum magnitude ξ of the propagator and the vertex time τ, and primes are derivatives with respect to x. Section 2.5 lists the presets. A massive propagator e with mass parameter ν_e and the `"h"` preset $P_{\mathrm{h}}(x) = (2\nu_e + 1)/x$, $Q_{\mathrm{h}}(x) = 1$ has Hankel blocks at each of its two vertices that span a two-dimensional space. The space is indexed by the state $n_{e,i}$, where $i = 1, 2$ names the endpoint of e, and $a_{e,i}$ denotes the time power of that same endpoint. For a cycle propagator, the second-order equation of motion at $n_{e,i} = 2$ gives the exact relation

$$\begin{aligned} J\big[\cdots, n_{e,i} = 2, \cdots\big] = & -J\big[\cdots, n_{e,i} = 0, \cdots\big] \\ & -(2\nu_e + 1)\, J\big[\cdots, n_{e,i} = 1,\, a_{e,i} \to a_{e,i} - 1,\, b_e \to b_e + 1, \cdots\big], \end{aligned} \tag{2.3}$$

so every massive vertex state reduces to the representative set $\{0, 1\}$. For a fixed propagator the second term instead carries an explicit factor ξ_e^{-1} and only shifts $a_{e,i} \to a_{e,i} - 1$; there

is no b_e label to shift. The package applies this reduction by construction: the generated seeds contain only $n_{e,i} \in \{0, 1\}$.

The zero points belong to the declaration. A case supplies them through the key `zeroPointRules`, as rules for $a0_v$ and $b0_e$. Two settings of this key are used by the released examples. The sunrise and the mixed case keep them as free regulator symbols, for example $a0_v = \alpha_v$ and $b0_e = \beta_e$. The pure massive bubble fixes them instead: for a propagator of mass parameter ν under the `"h"` preset it takes $b0_e = -2\nu$, and at the vertices it takes $a0_v = 2\nu + \texttt{analyticRegulator}$. The last symbol is kept during seed generation and set to zero afterwards. It is what the initialisation audit of section 3.1 asks for when the small-time leading powers of a family reach a divergent time-integral boundary under integer index shifts. With the zero points fixed this way, the labels $a_v, b_e, n_{e,i}$ that the package manipulates run over small integers. They make the IBP shifts uniform: the generators shift the package-level labels by ± 1 and never touch the zero points.

Massless quotient. A massless full propagator e between the ordered vertices u and v has two independent states. Let $\Delta = \tau_u - \tau_v$ and let q be the magnitude of its momentum. Its kernel is built from the two functions

$$\begin{aligned} M_0 &= \theta(\Delta)\, e^{-\mathrm{i}\sigma q\Delta} + \theta(-\Delta)\, e^{+\mathrm{i}\sigma q\Delta}, \\ M_1 &= -\theta(\Delta)\, e^{-\mathrm{i}\sigma q\Delta} + \theta(-\Delta)\, e^{+\mathrm{i}\sigma q\Delta}, \end{aligned} \tag{2.4}$$

where σ is the common branch sign of the two endpoints, so $\sigma = +1$ on $++$ and $\sigma = -1$ on $--$. Exchanging the two vertices leaves M_0 unchanged and flips the sign of M_1. Write $F_{n_1 n_2}$ for its propagator block with $(n_{e,1}, n_{e,2}) = (n_1, n_2)$. The package works with the quotient

$$F_{00} = M_0, \qquad F_{10} = M_1, \qquad F_{01} = -F_{10}, \qquad F_{11} = -F_{00}, \tag{2.5}$$

so only the two representatives F_{00} and F_{10} are independent. The pack of such a propagator still keeps both state powers, and each of them takes the values 0 and 1. Seeds are generated for the two representatives only, and a state produced by a derivative is reduced back with $n_{e,2} \to 0$. This is an exact algebraic identification, not a numerical sampling rule.

Contact terms and sector normalisation. Differentiating the time-ordering factors produces contact terms, also called pinching or shrinking terms in refs. [2–4]. For the default massive and massless full blocks, a nonzero elementary contact requires $n_{e,1} + n_{e,2} = 1$. The package merges the endpoints, removes the corresponding block, and maps the time powers, momentum powers and zero points to the child sector. For a cycle propagator in the default `"h"` system, for example, the merged time label is $a_u + a_v - 1$, its zero point is $a0_u + a0_v - 2\nu_e$, and the residual momentum labels are $b_{S,e} = b_e + 1$ and $b_{S0,e} = b0_e + 2\nu_e$. For a massless contact the time powers add without these shifts, and $b_{S,e} = b_e$.

Parallel full propagators joining the same pair of active vertices share a time-ordering boundary. They must be differentiated as one product. For each such bundle B, the contact expansion contains nonempty odd subsets $Q \subseteq B$ with combinatorial factor $2^{1-|Q|}$. Each term merges the endpoint pair once. If c^{raw}_{sQ} is its coefficient in the unnormalised

integral convention, including the combinatorial and elementary contact factors, then its contribution in the public convention is

$$\sum_{\substack{\emptyset\neq Q\subseteq B\\ |Q| \text{ odd}}} c_{sQ}^{\mathrm{raw}}\,\frac{N_s}{N_{t_Q}}\,J_{t_Q}\,. \tag{2.6}$$

Here t_Q denotes the child sector with all index and zero-point shifts applied. The ratio of normalisations is essential: the child normalisation already contains the assigned contraction constants and no-loop magnitude powers. The package applies this conversion when assembling IBP relations, so these factors must not be multiplied into the public integrals a second time.

2.3 Kinematics

The structural loop number L is the first Betti number of the internal undirected multigraph. Let N_E count the internal propagators, each one counted once: parallel propagators between the same pair of vertices therefore contribute separately, and a self-loop contributes one. External legs do not contribute to N_E. Let N_V count the vertices and C the connected components. Then

$$L = N_E - N_V + C\,. \tag{2.7}$$

Together with K independent loop-related external momenta, the loop number fixes the number of scalar products,

$$N_{\mathrm{sp}} = \frac{L(L+1)}{2} + LK\,. \tag{2.8}$$

The user declares two separate lists. The first list, `loopExternalMomenta` $\{k_1,\dots,k_K\}$, defines the loop coordinates `ss`$ij = \sqrt{\mathrm{sp}[k_i,k_j]}$ of the loop Gram matrix. The second list, `independentExternalMomenta`, defines the no-loop magnitudes |`kE`i| that appear in vertex energies and in the masses of fixed propagators. These magnitudes are parameterised by `sE`i. Only magnitudes are coordinates. The package never outputs $\mathrm{sp}[k_{Ei},k_{Ej}]$, and it never eliminates intermediate magnitudes by a cosine rule. Magnitudes that depend on the declared ones are stored as explicit bindings. They still contribute to vertex phases through the chain rule.

User-defined ISPs are declared as `name/expr/range` associations, for example

```
<|"name" -> rho1, "expr" -> sp[q1, k], "range" -> {0, 1}|>,
```

and they must form an invertible coordinate system on the space of scalar products together with the propagator squares. The definition zero point of an ISP is fixed at 0: positive powers are numerator powers, and a negative power means an extra denominator.

The kinematic coordinates of a case are fixed by rules that express the basic scalar products in terms of chosen symbols. A rule has the form `sp[k1,k1] -> u^2`, and a rule can bind a whole energy as well, as `sp[k1,k1] -> (E0-sE2)^2` does in the mixed example of section 4. The set of these rules is the kinematic system of the case, and its symbols are exactly the variables that later derivatives and scaling checks use. The two-stage interface `DSKinematics`/`DSInit` validates this system and reports an exact, over-complete or under-complete state. Only the exact state opens derivative generation.

2.4 Parity structure

For the massive `"h"` and `"H"` presets and the massless systems, the discrete invariant of the dS IBP system is the parity of $b_e + n_{e,1} + n_{e,2}$ on a propagator. The package does not automatically extend this parity support to general custom function systems. For every propagator e that carries a propagator power b_e, that is for every propagator of the cycle skeleton, the quantity

$$p_e \;=\; b_e + n_{e,1} + n_{e,2} \pmod 2 \tag{2.9}$$

is conserved by every time generator and every loop-momentum generator. A massless cross propagator has $n_{e,1} = n_{e,2} = 0$, so its parity is just b_e. A fixed propagator carries no propagator power, so it enters no parity label, and the package rejects a constraint that names one. The parity vector $(p_e)_e$ of the top sector labels the closed subsystems of the IBP system of a family, and for a graph whose propagators all lie on the cycle skeleton this is exactly the split stated by ref. [4].

The default `"h"` and `"H"` shrinks of a massive propagator, with the contact mapping of section 2.2, leave the parity unchanged: such a shrink raises the power to $b_e + 1$ and removes the single state power it consumes. A massless contact keeps the power at b_e, so it offsets the parity of the child sector by one, and several contacts accumulate modulo two. The package therefore does not apply a constraint to a sector below the top sector in its original form. It rewrites the constraint in the powers of that sector, using the power and zero-point shifts of its shrunk propagators. When the rewrite is not integral, the parity filter is disabled.

For families with user-provided `parityConstraints` the package keeps only the seeds whose transported constraint vanishes. A constraint is written in the powers of (2.9), for example $b_1 + n_{1,1} + n_{1,2} \equiv 0$ for the propagator with declaration index 1; the declaration spells the same powers as `b[1]`, `n[1,1]` and `n[1,2]`. ISP indices receive no parity filter.

2.5 Massive function systems

Besides the default `"h"` system defined in section 2.2, the package accepts bare Hankel functions, the preset `"H"`, and general user systems (P, Q, T, W) whose coefficients admit the finite Laurent-shift representation used by the generators and whose Wronskian data are consistent. Let T be an invertible 2×2 basis transformation acting on $(f, f')^{\mathsf{T}}$, and let W be the Wronskian before that transformation. The package tracks the combination $W_T = \det(T)\, W$ through all shrinks. The θ-coincidence and Wronskian shrink terms used by the time-IBP generators are derived from it. Ordinary derivatives of the kinematic coordinates never generate θ shrinks and never invoke W_T. The user chooses the convention once at initialisation. The IBP generators, the derivative operators and the tree-level track all read the same compiled function system.

2.6 Tree-level and time-only families

For graphs without cycles the package exposes a `timeOnly` family with the unique public representation

```
J[sectorKey_String, timeShifts_List, stateBits_List]
```

Here the *root topology* is the declared graph before any contact, and its propagator order is fixed for the whole family. The `sectorKey` records, for each propagator of the root topology, whether it is full (1) or shrunk (0). The list `timeShifts` gives the time powers a_v of the active vertices. The list `stateBits` gives one bit for each massive vertex state or massless quotient state, in the fixed order of the root propagators. There are no momentum generators, no propagator powers and no ISPs. The magnitude powers of fixed propagators remain in N_s and still enter derivative generation. This mode is the default for acyclic graphs. It can also be selected explicitly for the time-integral part of a graph with cycles, with its internal momentum magnitudes treated as parameters rather than integration variables. In the full loop mode, a non-cyclic propagator remains a fixed propagator in the three-slot family of section 2.2. The direct tree-formula interfaces currently support massive-only families. For a context containing any massless propagator, they return `PendingRederivation` with reason `masslessQuotientFormulaNotCertified`. The general `DSSeeds`, `DSGenerateIBP` and `DSLinear` route remains available for these families.

3 Using the package

The loop workflow has three stages: construct the family and its IBP system, obtain an external reduction, and use it to build differential equations. The commands below illustrate these stages; complete runnable declarations and scripts are supplied with the examples in section 4. Load the distributed `dSIBP.m` with `Get[packageFile]`, where `packageFile` is its full path. The `main.wl` script of example 04 supplies the massive-bubble declaration used below, under the name `caseInput`; `case` below stands for such an input association, and `custom` for its custom kinematic rules, declared as `kinematicRules` in example 06. Options follow the Wolfram convention, and `Options[function]` lists them at run time. Two options are shared by several commands and are not repeated below: `ProgressReporting` (`Automatic` by default) switches the console progress of the long-running commands on or off, and `AuditLevel` (`"standard"` by default; `"full"` re-computes the development certificates) sets the audit depth of `DSGenerateIBP`, `DSLinear` and the tree commands `DSTreeNaiveIBP`, `DSTreeNaiveDE` and `DSTreeDLogDE`. The remaining options and specification keys are introduced together with the commands that accept them.

3.1 Initialisation

A case is the named declaration described in section 2.2. Initialisation runs a topology audit of the loop number, the registry of vertices and propagators and the external-momentum routing. It then runs the kinematic audit of section 2.3. It also certifies the time-integration domain: if an integer shift of the continuous labels drives the leading small-time powers of a vertex to a divergent boundary, the run stops and names the vertices that need a symbolic regulator. The result is a context that every later command consumes:

```
context = DSInit[case, KinematicRules -> custom];
```

The optional two-stage interface `DSKinematics[case]` first returns a proposal with the default rules and a selection template. The user can then inspect the kinematic coordinates

`ss`ij, `sE`i, the momentum-routing audit and the capability flags before committing, and `DSKinematics[case, rules]` audits a chosen set of rules in the same form. Under-complete declarations abort the initialisation. Over-complete ones allow symbolic seed generation but disable derivatives and the inner-form rewriting. In the over-complete case common loop shifts are quotiented out through an affine basis `effectiveLoopExternalMomenta`.

Initialisation accepts the dedicated options

- `KinematicRules`: the custom rules above;
- `GenerateDerivativeMetadata` (`False` by default): pre-compute the metadata of the family's derivative operators, which the written initialisation files then include;
- `WriteInitializationFiles` and `InitializationDirectory` (`False` and `Automatic` by default): write the parsed topology, sectors and conventions as reloadable files;
- `OverwriteInitialization` (`False` by default): a directory holding a different input is refused unless this option is set;
- `RegisterAsCurrent` (`True` by default): `False` initialises without replacing the current context, as `DSRedefineParameters` does internally.

The command `DSRedefineParameters` itself re-initialises the same input with a complete new rule set and carries no options beyond the shared ones.

3.2 Seeds and the IBP index ranges

The sectors of a family are ordered by their contact operations. The *top sector* retains all propagator blocks; lower sectors arise by shrinking blocks and identifying their endpoints. The labels of the top sector are the root labels used below. `DSSeeds[context]` returns the canonical seed templates of the family, and `DSAllSeeds` collects them into a list:

```
seedData = DSSeeds[context];
allSeeds = DSAllSeeds[seedData];
```

Every sector receives all the time generators of its active vertices and all $L(L+K)$ loop-momentum generators. The massive vertex states are pre-reduced by the equation of motion (2.3). Massless full propagators run over the two quotient representatives of (2.5). The discrete state powers are eliminated, and the analytic seeds keep general continuous labels $a_v, b_e, b_{S,e}$ and general ISP indices. The canonicalisation applies the symmetry rules of the declaration, where the user names a vertex swap or an exchange of parallel propagators, together with built-in tadpole rules. A tadpole rule is available only for a self-loop whose reversal is a single loop-momentum inversion. It maps the integral to zero when an ISP carries an odd power and when a massless full propagator sits in the F_{10} representative; a massive full propagator with $(n_{e,1}, n_{e,2}) = (1, 0)$ exchanges its two endpoint states.

The continuous labels are swept by `DSGenerateIBP`. The unified form

```
ibpData = DSGenerateIBP[allSeeds, {-1, 1}];
```

requires that every root continuous index lies in the closed integer interval $[-1, 1]$ *after* all shifts of the generated relations. The package inverts the per-index shift sets and derives the narrower seed ranges automatically. The per-index form

```
ibpData = DSGenerateIBP[allSeeds,
   {a[v1], -1, 1}, {a[v2], 0, 1},
   {b[e1], 0, 2}, {b[e2], -1, 1}];
```

must cover all root continuous indices exactly. ISP lower bounds are combined with the user target by taking the larger lower bound, so the package never extends a range on its own in order to cover numerator terms. Only complete systems may proceed to a formal reduction. Neither `DSSeeds` nor `DSGenerateIBP` carries options beyond the shared ones.

3.3 Linear systems and external reduction

The command

```
linearData = DSLinear[ibpData, context];
```

converts the generated IBP relations into a backend-neutral association. Its fields include the equations `linearEquations`, the full integral list `integralList`, the integral-to-ID mapping `integralRules`, and sector metadata. `DSLinear` accepts the dedicated options `LinearSystemMode` (`"symbolic"` by default; `"numeric"` builds the sampled system with the numeric rules substituted), `CoefficientRules` (`Automatic` by default; the numeric substitutions of the sampled mode) and `KiraOrdering` (`Automatic` by default; an association that adjusts the integral-ordering conventions of the serializer). The command `DSReorderIntegrals` changes this ordering before export. Master integrals are identified at the subsequent reduction stage. The two-stage Kira plan is built in memory:

```
prePlan   = DSKiraPlan[linearData, <|
   "stage" -> "preReduction", "candidateIntegrals" -> candidates,
   "outputDirectory" -> preDir, "jobOptions" -> Automatic|>];
preExport = DSKiraExport[prePlan];
(* ... external Kira run identifies the master integrals ... *)
linearWithUserMI = DSUserMI[linearData, userExpressions, <|
   "names" -> activeNames, "activeIndices" -> activeIndices,
   "derivativeVariables" -> {ss11, P0}, "scalingDegrees" -> degrees|>];
formalPlan   = DSKiraPlan[linearWithUserMI, <|
   "stage" -> "formal", "activeBasis" -> Automatic,
   "numericStage" -> "symbolic"|>];
formalExport = DSKiraExport[formalPlan];
(* ... external Kira run reduces to the user basis ... *)
importedData = DSKiraImport[formalWorkspace];
```

The specification of `DSKiraPlan` must contain `"stage"` and accepts the further keys

- `"preferredIntegrals"`: integrals recorded as preferred candidates;

- `"candidateIntegrals"`: the pre-reduction targets, the full integral list by default;
- `"activeBasis"`: the basis attached by `DSUserMI` when left `Automatic`;
- `"numericStage"`: `"symbolic"` by default, `"postDerivative"` substitutes the numeric rules only after the analytic derivative closure is frozen;
- `"coefficientRules"` and `"outputDirectory"`: forwarded to the export;
- `"jobOptions"`: the Kira job configuration, for example the parallel job count.

The pre-reduction stage freezes the candidate targets and lets the external reducer discover the masters. The command `DSUserMI` then attaches the user basis. Its specification keys `"names"` (defaulting to `"active1"`, `"active2"` and so on), `"activeIndices"` (defaulting to all positions), `"derivativeVariables"` and `"scalingDegrees"` (both defaulting to `Automatic`) are optional. It checks the global and the active rank, and it stores reversible pivot rules. The formal stage constructs the analytic derivative closure of the active basis *before* any export. `DSKiraExport` writes the input directories of a plan, or of plain `linearData`; its options are

- `OutputDirectory` (`None` by default);
- `KiraTargetIntegrals` (`Automatic`): the mandatory targets of the run;
- `KiraCoefficientRules` (`{}`): the numeric coefficient rules;
- `KiraJobOptions` (`Automatic`): the job configuration;
- `KiraActiveBasis` (`None`): the active basis of a formal export;
- `KiraRequireCompleteSystem` (`True`): refuse an incomplete system;
- `KiraNumericStage` (`"symbolic"`).

A plan supplies the corresponding values from its fields. `DSKiraImport` checks the export identity when importing the reduction. Its dedicated options `KiraReductionFile` and `KiraMasterFile` (`Automatic` by default) resolve to the Kira family outputs of the workspace, and `KiraCompletionFile` and `KiraCompletionPatterns` (`Automatic` by default) control the completion diagnostics of the run log. Kira accepts only real rational coefficients, so the serialiser rewrites the momenta that must lie on the imaginary axis by the backend rule `k -> -I ik`, where `I` is the imaginary unit and `ik` is the backend symbol for the same magnitude. The importer reverses this map to restore the physical variables and their derivative Jacobians. The current export/import interface is for Kira. The backend-neutral `linearData` representation also provides the input needed to develop serialisers for other reduction programs.

3.4 Differential equations and checks

After the reduction, `DSDE` differentiates the master integrals with respect to the independent variables:

```
deData = DSDE[importedData, {ss11, P0}];
```

The variable list defaults to all initialised variables. The option `OutputDirectory` (`None` by default) writes the DE data to disk. Loop-Gram coordinates are handled through external-vector derivative decompositions. No-loop magnitudes are differentiated through radial derivatives that are local to each propagator, including their dependent bindings. Vertex energies enter as scalar parameters. Explicit coefficients are differentiated directly, while bare integrals are shifted and canonicalised. The result is reduced back to the chosen active basis. The pure massive bubble example supplies a known $d\log$ basis from ref. [4]. Constructing the DE in that basis does not require a general algorithm for finding a $d\log$ transformation.

Under a common rescaling of momentum magnitudes and vertex energies, each integral has a definite degree. Let $x_1, \ldots, x_m$ denote these dimension-one coordinates. The spatial dimension and the mass parameters ν_e are held fixed under this rescaling; they may enter the scaling degrees. For a basis of homogeneous master integrals, each I_α satisfies the Euler relation

$$\sum_{j=1}^{m} x_j \frac{\partial I_\alpha}{\partial x_j} \;=\; d_\alpha \, I_\alpha \,, \tag{3.1}$$

where d_α is the constant scaling degree of I_α. In matrix form, with $\partial I/\partial x_j = \Omega_j(x)\, I$ for the vector of masters, the identity (3.1) becomes

$$\sum_{j=1}^{m} x_j \, \Omega_j(x) \;=\; \Delta \,, \qquad \Delta = \mathrm{diag}(d_1, \ldots, d_R) \,, \tag{3.2}$$

where R is the number of masters in the active basis. The command `DSScaleCheck` checks this matrix identity and any inhomogeneous source terms in the DE output. It takes the DE data and an optional specification whose keys `"relation"`, `"variables"`, `"weights"` and `"degrees"` are equivalent to its dedicated options `ScalingRelation` (`"Custom"` by default; the built-in relations `"PureMassiveBubble"` and `"LoopTopology"` supply the degrees of those families when none are declared), `ScalingVariables` (`Automatic`, the DE variables), `ScalingWeights` (`Automatic`, all weights one) and `ScalingDegrees` (`Automatic`, the degrees declared with the active basis). It reports a symbolic check when the scaling variables remain symbolic, a fixed-section check when some are fixed, and an exact-point check when all are fixed. The source-level time and momentum IBP checks quoted in section 4 are separate calculations in the validation scripts. The tree workflow exposes `DSTreeDLogDE` for direct $d\log$ differential equations.

At the atomic level the package also exposes the generators themselves:

```
dtau[i, expr, context]     (* time generator at vertex i *)
dqq[i, j, expr, context]   (* d/dq_i . q_j *)
```

```
dqk[i, j, expr, context]   (* d/dq_i . k_j *)
ds[expr, var, context]     (* full variable derivative *)
```

together with the form rewriters `rep2innerform`, `rep2outform` and `rep2Integrand`. In these four calls the first argument of `dtau` is a vertex index, and the first two arguments of `dqq` and `dqk` are loop-momentum and external-momentum indices. The last command expands a J into a formal integrand retaining named propagator blocks, without integrating. The short signatures rely on the single registered topology. Explicit contexts should be used when several topologies are registered.

3.5 Tree-level track

The tree-level functions mirror the loop workflow:

```
treeSeeds = DSTreeSeeds[integral, context];
dlogDE    = DSTreeDLogDE[context];
iterative = repIterative[target, end, context];
naiveIBP  = DSTreeNaiveIBP[context, masters];
naiveDE   = DSTreeNaiveDE[naiveIBP, variables];
```

Here `integral` is a member of the `timeOnly` family, and `end` fixes the target time powers of the iterative reduction, defaulting to the zero power at every vertex. The `masters` argument of `DSTreeNaiveIBP` defaults to the master list of `dlogDE` in the same order, and the `variables` argument of `DSTreeNaiveDE` defaults to the vertex energies and massive-leg magnitudes of the family. `DSTreeNaiveDE` also accepts the result of `DSTreeNaiveIBP` directly. `DSTreeSeeds` can be called for a single vertex or, as above, for every time-generator vertex at once; the tree commands carry no options beyond the shared ones. The iterative route substitutes the general reduction formulas, whereas the naive route solves sampled IBP relations. The two-vertex example compares the naive DE with `DSTreeDLogDE`. Section 4.1 describes this check and the separate comparison with MadStree. Figure 3 summarises the two workflows and their connection to the external reducers. Table 1 lists the public functions.

4 Examples

The package ships six worked examples. Figure 4 shows the four graphs discussed below. The quoted Kira times are wall-clock times of the external reduction on a single workstation. The Kira inputs were written by `DSKiraExport`.

Pure massive bubble (example 04). The bubble of figure 4b has two vertices on the $-$ branch. Its two massive full propagators carry the same mass parameter ν and the loop momenta q and $q-k$. The loop external momentum k gives the loop coordinate $\texttt{ss11} = |k|$. Both vertex energies are set to the parameter P_0, which is the second derivative variable. This example covers the full workflow: seeds, index-range sweep, linear system, two-stage Kira export, external reduction, import, differential equations and scaling check. The Kira workspace must lie outside the repository, and the package itself never starts

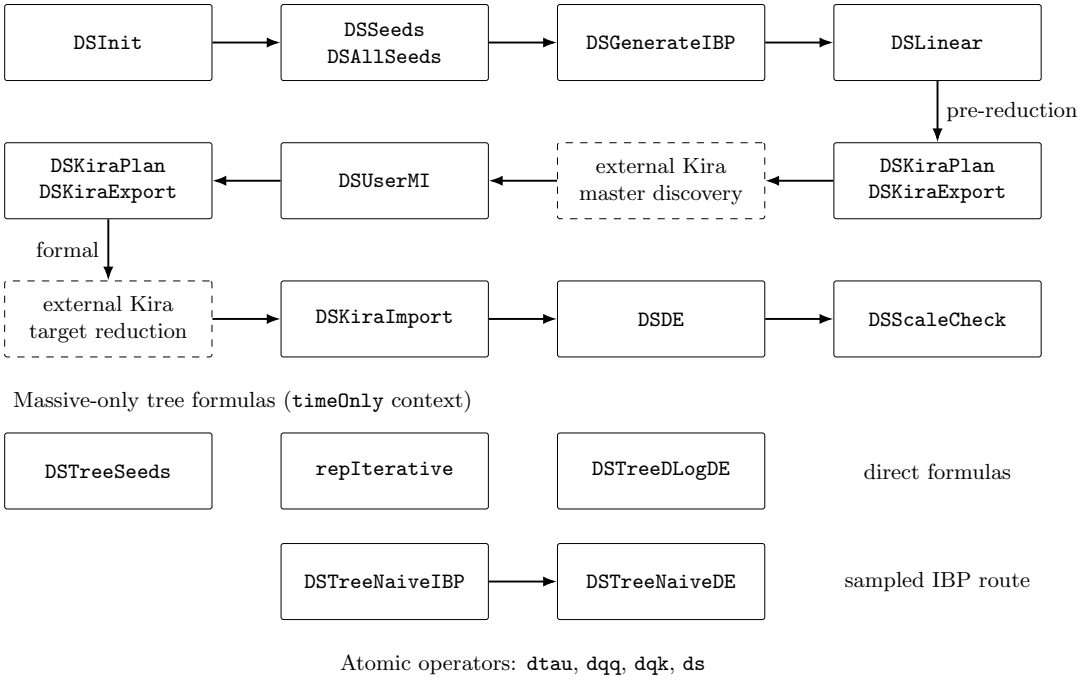


Figure 3. The public workflow of `dSIBP`. Full (loop) families run from `DSInit` through the seed, index-range and linear-system stages to the two-stage Kira export and import and to the differential-equation layer. The command `DSUserMI` attaches the user basis between the two reduction stages. For supported massive-only tree families, `DSTreeSeeds`, `repIterative` and `DSTreeDLogDE` expose the direct formulas; the naive IBP route provides a separate check. The atomic operators are listed below the workflows.

Kira. The linear system is written at a fixed rational probe point for the dimension and the mass parameters, while the differential equations keep `ss11` and P_0 symbolic. The reduction terminates with no unreduced target and returns nineteen master integrals. The two 19×19 differential-equation matrices in $d\log$ form agree with those of ref. [4] at the recorded exact point. The scaling identities (3.2) hold symbolically on the same basis. Section 4.1 gives the comparison conventions and check counts.

Mixed bubble plus tree (example 06). The largest released case is figure 4c. Propagators 1 and 2 form the loop: propagator 1 is a massive full propagator with mass parameter ν_1, and propagator 2 is a massless full propagator. Propagator 3 is a massless fixed propagator, a bridge, between v_2 and v_3. It lies outside the loop, so its pack carries no propagator power and its magnitude power is collected in N_s. One external leg enters v_1 and carries $k_1 + k_2$, and two external legs enter v_3 and carry k_1 and k_2. The vertex v_2 has no external leg. The declaration gives the three vertices the three independent energies E_1, E_2, E_3 of (2.1), and these are derivative variables in their own right. The loop external momentum is $k_1 + k_2$, which gives `ss11`. The independent no-loop momenta k_1, k_2 give the magnitudes `sE1`, `sE2`. Both cycle propagators are restricted to even parity, $b_e + n_{e,1} + n_{e,2} \equiv 0$ for $e = 1, 2$ in the labels of (2.9), while the fixed propagator is excluded

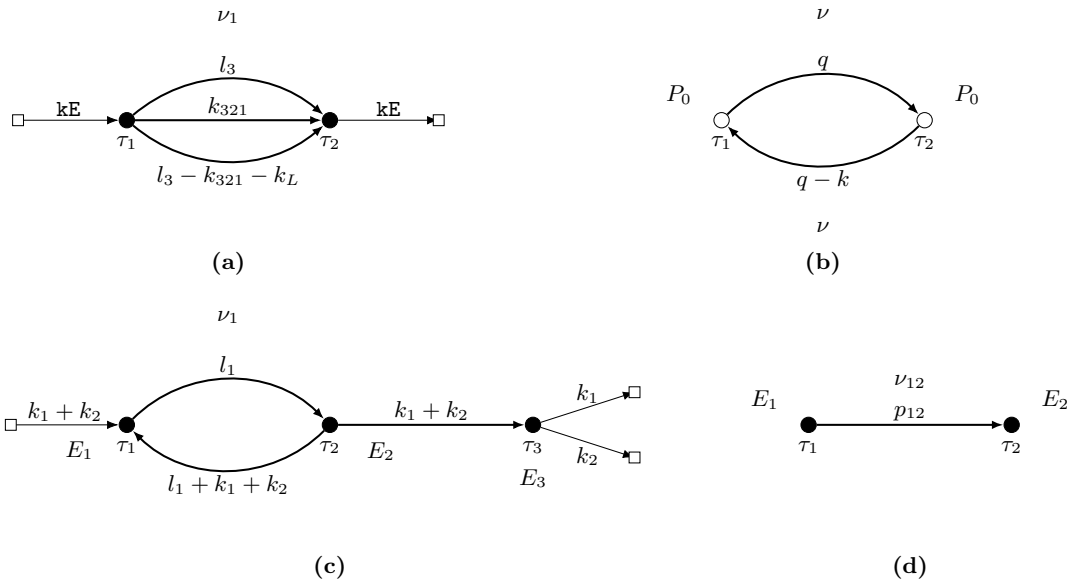


Figure 4. Example graphs. The fill colour of a vertex encodes its branch sign: black is + and white is −. (a) The two-loop sunrise of example 03: one massive full propagator with mass parameter ν_1 and two massless full propagators between two + vertices; both vertices declare the same vertex energy, written `kE`. (b) The pure massive bubble of example 04: two massive full propagators with the same mass parameter ν between two − vertices with vertex energy P_0; the loop external momentum is k. (c) The mixed bubble-plus-tree case of example 06: a massive full propagator with mass parameter ν_1 and a massless full propagator form the loop between v_1 and v_2. A massless fixed propagator, a bridge, connects v_2 and v_3, and the external legs carry $k_1 + k_2$, k_1 and k_2. (d) The two-vertex tree of example 05: a massive propagator with mass parameter ν_{12} whose momentum p_{12} is a no-loop magnitude.

from the parity constraints. The parity-restricted family has eighty-one master integrals, and the generated system contains 818,217 canonical relations. The formal Kira 2.3 reduction takes 160.11 s under WSL with ten threads and terminates with no unreduced target. This time excludes seed generation, basis discovery, export and post-processing. At the recorded exact point, the package produces six 81×81 connection-matrix values, one for each of `loopScale`, `legScale1`, `legScale2`, E_1, E_2, E_3. The first three are the chosen names for the loop and two no-loop magnitude coordinates. No unreduced integrals or backend helper symbols remain. All 81/81 source-level Euler and IBP identities vanish exactly. The reduction-based scaling certificate applies at that point: all six scaling variables are fixed in the formal plan, so these results do not establish a symbolic six-variable DE.

The example also illustrates how declarations and kinematic redefinitions interact. One input assigns the energy E_0 to v_3 while retaining the two external legs k_1, k_2. After `DSInit`, `DSRedefineParameters` imposes $E_0 = |k_1| + |k_2|$ through `sp[k1,k1] -> (E0-sE2)^2` and `sp[k2,k2] -> sE2^2`. A separate input replaces those two legs by one leg p_0 and assigns energy E_0 to v_3. It is initialised as a new topology before applying `sp[p0,p0] -> E0^2`. The example also runs the over-complete and under-complete kinematic audits.

Sunrise (example 03). The graph of figure 4a has two vertices joined by three parallel propagators, hence $L = 2$. Propagator 1 is a massive full propagator with mass parameter ν_1, and propagators 2 and 3 are massless full propagators. The loop momenta are l_3 and k_{321}, and the single loop external momentum is k_L. Thus $K = 1$, and the loop Gram matrix gives the coordinate $\texttt{ss11} = |k_L|$. No no-loop magnitude is declared. Both vertices declare the same vertex energy, written `kE` in the case, and this is the second derivative variable. The top sector receives two time generators and $L(L + K) = 6$ loop-momentum generators. Two ISPs are declared, $\mathrm{sp}[k_{321}, l_3]$ and $\mathrm{sp}[l_3 - k_{321} - k_L, l_3]$, each with range $\{0, 1\}$. The symmetry that swaps the two massless propagators exchanges them, and the package canonicalises the two ISPs together with that swap. The example stops after the general stage. It generates the canonical seeds and the general parameter-derivative operators, and it neither samples the continuous labels nor runs a reduction, a differential equation or a scaling check. The example demonstrates parameter derivative generation by applying `ds` to the top-sector integral with respect to both variables.

Two-vertex tree (example 05). A massive propagator with mass parameter ν_{12} connects two + vertices, figure 4d. There is no loop momentum. The momentum p_{12} of this propagator is declared as an independent no-loop momentum, so the package builds the coordinate $\texttt{sE1} = |p_{12}|$. The `timeOnly` family has the two sector keys `"1"` and `"0"`, for the full and the shrunk propagator. The naive IBP route and the direct formula route give the same $d\log$ differential equations in $E_1, E_2, \texttt{sE1}$. The example separately checks that `repIterative` returns without failure. The same example also declares a massless family with a single propagator. Its seeds are available over the two quotient representatives F_{00} and F_{10} of (2.5), while its formula route remains pending as described in section 2.6.

Mixed bubble workflow and bare Hankel system (examples 01, 02). Example 01 declares a bubble with one massive and one massless full propagator between two + vertices. It runs the unified public entry through the discrete templates, the continuous sweep and the linear system. It builds the pre-reduction plan in memory without exporting or running Kira. Example 02 supplies a bare Hankel function system (P, Q, T, W) instead of the `"h"` preset. The package compiles it through the same public workflow and generates the seeds of the corresponding family. Together the two examples cover the initialisation, the index-range sweep and the function-system interface of sections 3 and 2.5.

4.1 Validation

The checks below test different parts of the calculation. Comparisons with analytic reference matrices test the differential equations at specified points; agreement there is not a symbolic proof for all kinematics. Source-level identities and reduced-basis scaling checks provide additional consistency tests.

1. *Analytic tree reference.* For a massive two-vertex family, the general IBP seeds and DE are compared with ref. [3], using its ordering of five normalised masters. Each of the three 5×5 matrices agrees in all 25 entries at the recorded probe point. A

separate comparison reduces sampled raw IBP relations of the same family and checks agreement with the general formulas. These tests cover IBP relations and DE.

2. *Loop reductions and scaling.* The pure massive bubble of example 04 combines an external Kira reduction to nineteen masters with a comparison against the supplementary matrices of arXiv v3 of ref. [4]. The benchmark directory of the package registers their SHA-256 identity. The reference construction orients the energy oppositely, $P_{\text{ref}} = -P_0$. At the recorded exact point, all 361/361 entries agree in each of three derivative conventions: the physical P_0, the backend variable `ip0` related by $P_0 = -\mathrm{i}\,\texttt{ip0}$, and $k_s = \texttt{ss11}$. The scaling identities (3.2) hold symbolically on the same basis, with 53/53 source-level identities and 19/19 active-basis checks. The mixed bubble-plus-tree family of example 06 checks reduction closure and the exact vanishing of all 81/81 source-level Euler and IBP identities, and verifies scaling on its eighty-one-master reduced basis at the recorded point. All six scaling variables are fixed in its formal plan, so this certificate does not establish a symbolic six-variable DE. These examples test the IBP-to-DE workflow and scaling identities. The mixed-family checks are consistency tests, rather than a comparison with an independently known analytic solution.

3. *Comparison with MadStree.* For a mixed three-vertex tree with fifteen masters, `dSIBP` parameter derivatives are reduced using MadStree [21]. The resulting matrices are compared with MadStree's direct $d\log$ construction for two no-loop magnitudes and three vertex energies. All seventy-five derivatives close, and the five 15×15 matrices agree entry by entry at the comparison point. This checks the `dSIBP` derivative operators against a separate DE construction; the reduction in this test is performed by MadStree.

5 Summary and outlook

`dSIBP` makes the dS IBP/DE framework accessible through graph declarations and a common MATHEMATICA interface. It automates the construction of integral families, canonical seeds and linear systems, and uses externally obtained loop reductions to construct DE and scaling checks. Equations of motion, massless state relations and parity constraints organise the system before reduction. The examples demonstrate the workflow for massive and mixed families, including a reduction to eighty-one masters.

The present implementation provides the algebraic input for subsequent evaluation of dS integrals. Extending the direct tree formulas to families with massless propagators remains an open implementation task. Further work concerns connecting the reduced DE to boundary-data constructions and numerical evaluation, building on the analytic methods of ref. [3] and tools such as MadStree [21].

Acknowledgements

We thank the developers of Kira [14, 15], Rational Tracer [16] and MadStree [21] for making their tools available.

A Public functions

Table 1 lists the public functions. The dedicated options and specification keys of each command are introduced together with that command in section 3, and `Options[function]` lists them at run time.

Table 1. Public functions of the package.

Function	Description
`DSInit`	initialise a case; topology, kinematic and time-domain audits; returns the context.
`DSInfo`	brief or full information about the initialised context.
`DSKinematics`	two-stage kinematic proposal and validation.
`DSParameterNotation`	inspect the current parameter notation.
`DSRedefineParameters`	re-initialise the context with a new complete set of kinematic rules.
`DSMessagesOn`, `DSMessagesOff`, `DSMessagesQ`	switch and query the optional messages; fatal errors always stay on.
`DSSeeds`	canonical seed templates of a family.
`DSAllSeeds`	flattened seed template list.
`DSSeedGroups`, `DSSeedGroupMetadata`	grouping of the templates by sector, class and generator.
`DSMetaSeedRange`	per-group shift metadata for range planning.
`DSGenerateIBP`	sweep continuous indices; sealed IBP relations.
`DSLinear`	backend-neutral linear system (`linearData`).
`DSReorderIntegrals`	re-index the full integral ordering.
`DSKiraPlan`	in-memory pre-reduction or formal reduction plan.
`DSKiraExport`	write Kira input directories; manifest with SHA-256 identities.
`DSKiraImport`	read back external reduction results with identity checks.
`DSUserMI`	attach a user master-integral basis with rank checks.
`DSDE`	differential equations of the reduced basis.
`DSScaleCheck`	Euler scaling identities of the family.
`DSTreeSeeds`	`timeOnly` seed templates.
`repIterative`	tree route via the general derived formulas.
`DSTreeNaiveIBP`, `DSTreeNaiveDE`	tree route via raw sampled IBP.
`DSTreeDLogDE`	tree $d\log$ differential equations.
`dtau`, `dqq`, `dqk`, `ds`	atomic generators and the full variable derivative.
`rep2innerform`, `rep2outform`	notation rewriters between user and internal forms.
`rep2Integrand`	expand J into a formal integrand retaining named propagator blocks.
`symmetry`	apply the user and built-in tadpole symmetry rules of section 3.2.
`repSymmetry0`	the raw user symmetry rules of the topology input.
`DSPublicAPI`	the registry of public functions used by this table.

References

[1] X. Chen, Y. Wang and Z.-Z. Xianyu, *Schwinger-Keldysh Diagrammatics for Primordial Perturbations*, *JCAP* **12** (2017) 006 [1703.10166].

[2] J. Chen and B. Feng, *Towards systematic evaluation of de Sitter correlators via Generalized Integration-By-Parts relations*, *JHEP* **06** (2024) 199 [2401.00129].

[3] J. Chen, B. Feng and Y.-X. Tao, *Multivariate hypergeometric solutions of cosmological (dS) correlators by d log-form differential equations*, *JHEP* **03** (2025) 075 [2411.03088].

[4] J. Chen, B. Feng, Z. Qin and Y.-X. Tao, *Loop integrals in de Sitter spacetime: The parity-split IBP system and* d log*-form differential equations*, 2604.14549.

[5] K.G. Chetyrkin and F.V. Tkachov, *Integration by parts: The algorithm to calculate β-functions in 4 loops*, *Nucl. Phys. B* **192** (1981) 159.

[6] S. Laporta, *High-precision calculation of multiloop Feynman integrals by difference equations*, *Int. J. Mod. Phys. A* **15** (2000) 5087 [hep-ph/0102033].

[7] A.V. Kotikov, *Differential equations method: New technique for massive Feynman diagrams calculation*, *Phys. Lett. B* **254** (1991) 158.

[8] A.V. Kotikov, *Differential equation method: The Calculation of N point Feynman diagrams*, *Phys. Lett. B* **267** (1991) 123.

[9] Z. Bern, L.J. Dixon and D.A. Kosower, *Dimensionally regulated pentagon integrals*, *Nucl. Phys. B* **412** (1994) 751 [hep-ph/9306240].

[10] T. Gehrmann and E. Remiddi, *Differential equations for two-loop four-point functions*, *Nucl. Phys. B* **580** (2000) 485 [hep-ph/9912329].

[11] J.M. Henn, *Multiloop integrals in dimensional regularization made simple*, *Phys. Rev. Lett.* **110** (2013) 251601 [1304.1806].

[12] A.V. Smirnov and F.S. Chukharev, *FIRE6: Feynman Integral REduction with modular arithmetic*, *Comput. Phys. Commun.* **247** (2020) 106877 [1901.07808].

[13] A.V. Smirnov and M. Zeng, *FIRE 7: Automatic Reduction with Modular Approach*, *Comput. Phys. Commun.* **325** (2026) 110200 [2510.07150].

[14] P. Maierhöfer, J. Usovitsch and P. Uwer, *Kira—A Feynman integral reduction program*, *Comput. Phys. Commun.* **230** (2018) 99 [1705.05610].

[15] J. Klappert, F. Lange, P. Maierhöfer and J. Usovitsch, *Integral reduction with Kira 2.0 and finite field methods*, *Comput. Phys. Commun.* **266** (2021) 108024 [2008.06494].

[16] V. Magerya, *Rational Tracer: a Tool for Faster Rational Function Reconstruction*, 2211.03572.

[17] M. Hidding, *DiffExp, a Mathematica package for computing Feynman integrals in terms of one-dimensional series expansions*, *Comput. Phys. Commun.* **269** (2021) 108125 [2006.05510].

[18] X. Liu, Y.-Q. Ma and C.-Y. Wang, *A Systematic and Efficient Method to Compute Multi-loop Master Integrals*, *Phys. Lett. B* **779** (2018) 353 [1711.09572].

[19] X. Liu and Y.-Q. Ma, *AMFlow: A Mathematica package for Feynman integrals computation via auxiliary mass flow*, *Comput. Phys. Commun.* **283** (2023) 108565 [2201.11669].

[20] Z.-F. Liu and Y.-Q. Ma, *Determining Feynman Integrals with Only Input from Linear Algebra*, *Phys. Rev. Lett.* **129** (2022) 222001 [2201.11637].

[21] "MadStree: automated numerical evaluation of tree-level de Sitter correlators." in preparation, 2026.